Guia Completo: Meta Ads Dashboard com Vercel

Como criar um webapp de análise de anúncios usando a Meta Graph API, HTML puro e deploy serverless no Vercel — com lições aprendidas, erros e soluções reais.

DATA	STACK	NÍVEL	TEMPO ESTIMADO
Março 2026	HTML · Vercel · Meta API	Iniciante → Intermediário	2–4 horas

Índice

1 Visão geral e tecnologias

2 Arquitetura da solução

3 Passo a passo: do zero ao deploy

3.1 Estrutura de arquivos

3.2 Criando o proxy Vercel

3.3 Configurando o dashboard

3.4 Deploy e variáveis de ambiente

3.5 Renovando o token

4 Erros encontrados e soluções

5 Métricas implementadas

6 Como economizar tokens do Claude

7 Próximos passos e melhorias

8 Checklist de deploy

1 Visão Geral e Tecnologias

O que foi construído, por quê, e com quais ferramentas.

O QUE É O PROJETO

Um webapp privado de análise de Meta Ads que consome a Meta Graph API v21 e exibe campanhas, anúncios, métricas criativas (Hook Rate, Hold Rate, CPC Link) e gráficos de evolução diária — acessível por qualquer pessoa da equipe via URL do Vercel.

Stack utilizada

TECNOLOGIA	PAPEL	POR QUÊ
HTML + Tailwind CDN	Interface / frontend	Zero build step, deploy simples
Chart.js	Gráficos	Leve, fácil via CDN
Vercel (Serverless)	Hospedagem + proxy API	Gratuito, deploy automático via GitHub
Node.js (api/meta.js)	Proxy server-side	Protege o token de acesso
Meta Graph API v21	Dados de anúncios	Fonte oficial dos dados
GitHub (privado)	Controle de versão	Integração automática com Vercel

Conta e IDs usados

Ad Account: `act_717055595505474` (CA - A Ordem)

API Version: `v21.0`

Base URL: `https://graph.facebook.com/v21.0/`

Repo GitHub: `grsgustao-hash/meta-ads-analyzer` (privado, fork)

2 Arquitetura da Solução

Como o fluxo de dados funciona, desde o browser até a Meta API.

FLUXO DE DADOS

🌐 Browser (dashboard.html)

| fetch("/api/meta?path=act_.../insights&...")

⚡ Vercel Serverless (api/meta.js)

| injeta META_ACCESS_TOKEN (env var segura)

| fetch("https://graph.facebook.com/v21.0/...")

📊 Meta Graph API v21

| retorna JSON com insights

🖥️ Dashboard renderiza os dados

⚠️ **POR QUE O PROXY É ESSENCIAL**

Se o token for colocado diretamente no `dashboard.html`, qualquer pessoa que inspecionar o código-fonte do browser consegue roubá-lo e usar sua conta de anúncios. O proxy garante que o token **nunca sai do servidor Vercel**.

Arquivos do projeto

ARQUIVO	FUNÇÃO
`dashboard.html`	Interface completa — HTML, CSS e JS em um só arquivo
`api/meta.js`	Proxy serverless — recebe requisições do browser e chama a Meta API com o token
`vercel.json`	Configuração do Vercel — redireciona / para dashboard.html

3 Passo a Passo: Do Zero ao Deploy

Siga esta sequência para replicar o projeto do zero.

3.1 — Estrutura de arquivos

```
meu-projeto/
├── dashboard.html      ← frontend
├── vercel.json         ← config Vercel
└── api/
    └── meta.js         ← proxy serverless
```

3.2 — Proxy Vercel (api/meta.js)

Este arquivo é o coração da segurança do projeto. Ele intercepta todas as chamadas do browser e injeta o token:

```
module.exports = async function handler(req, res) {
  res.setHeader('Access-Control-Allow-Origin', '*');
  res.setHeader('Access-Control-Allow-Methods', 'GET, OPTIONS');
  if (req.method === 'OPTIONS') return res.status(200).end();

  const token = process.env.META_ACCESS_TOKEN;
  if (!token) return res.status(500).json({ error: { message: 'META_ACCESS_TOKEN not set' } });

  const { path, ...rest } = req.query;
  if (!path) return res.status(400).json({ error: { message: 'Missing path' } });

  const params = new URLSearchParams({ ...rest, access_token: token });
  const metaUrl = `https://graph.facebook.com/v21.0/${path}?${params}`;

  try {
    const resp = await fetch(metaUrl);
    const data = await resp.json();
    return res.status(resp.status).json(data);
  } catch (err) {
    return res.status(500).json({ error: { message: err.message } });
  }
};
```

3.3 — vercel.json

```
{
  "routes": [{ "src": "/", "dest": "/dashboard.html" }]
}
```

3.4 — Helpers no dashboard.html

Toda chamada à API passa pelos helpers abaixo — nunca use a URL da Meta diretamente no browser:

```
// Constrói URL para o proxy
function apiUrl(path, params = {}) {
  return `/api/meta?${new URLSearchParams({ path, ...params })}`;
}

// Converte URL de paginação da Meta para o proxy
function toProxy(url) {
  if (!url.startsWith('http')) return url;
  const u = new URL(url);
  u.searchParams.delete('access_token');
  const path = u.pathname.replace(/^\/v[\d.]+\//, '');
  return `/api/meta?${new URLSearchParams({ path, ...Object.fromEntries(u.searchParams) })}`;
}

// Parâmetro de data (preset ou range personalizado)
function dateObj() {
  if (currentDateParam.preset) return { date_preset: currentDateParam.preset };
  return { time_range: JSON.stringify({ since: currentDateParam.since, until: currentDateParam.until }
}
```

3.5 — Deploy no Vercel

1. **Criar repositório no GitHub (privado)**

   Crie um repo privado no GitHub. Se for um fork de projeto público, fork → settings → make private. Use o GitHub Desktop para commits se não tiver git configurado no terminal.

2. **Conectar ao Vercel**

   Acesse `vercel.com` → New Project → Import Git Repository → selecione seu repo. O Vercel detecta automaticamente que é um projeto estático.

3. **Configurar variável de ambiente**

   No Vercel: Settings → Environment Variables → adicione:

```
Name:  META_ACCESS_TOKEN
Value: EAABsbCS... (seu token Meta)
Env:   Production, Preview, Development
```

4. **Fazer Redeploy**

   Após adicionar a env var, é necessário um novo deploy para ela ser aplicada: Deployments → → Redeploy.

3.6 — Renovando o token (a cada ~60 dias)

⏰ **QUANDO RENOVAR**

O dashboard vai mostrar o erro: *"Session has expired on..."* — é o sinal de que o token precisa ser renovado.

1. **Gerar novo token**

   Acesse `developers.facebook.com/tools/explorer` → selecione o app → Generate Access Token → marque `ads_read` + `ads_management`.

2. **Estender para 60 dias**

   Clique no ícone
   "i"
   ao lado do token → Open in Access Token Tool →
   Extend Access Token

.

3 **Atualizar no Vercel**

Vercel → Settings → Environment Variables → editar `META_ACCESS_TOKEN` → colar novo token → Salvar.

4 **Redeploy**

Deployments → → Redeploy.

✅ **SOLUÇÃO DEFINITIVA: SYSTEM USER TOKEN (NÃO EXPIRA)**

Para nunca renovar o token, crie um **System User** no Meta Business Manager:

1. Business Manager → Configurações → Usuários → Usuários do sistema
2. Criar System User (nome sem espaços, ex: `dashboard-api`) com papel Admin
3. Adicionar o System User ao app Meta em `developers.facebook.com` → App → Funções do app
4. Voltar ao Business Manager → System User → Gerar Token → selecionar o app → marcar permissões
5. Token gerado nunca expira — colocar no Vercel e esquecer

4 Erros Encontrados e Soluções

Todos os erros reais que apareceram durante o desenvolvimento, com causa e solução.

✕ Erro 1 — Campo deprecated na Meta API

```
(#100) video_3_sec_watched_actions is not valid for fields param
```

Causa: A Meta deprecou o campo `video_3_sec_watched_actions` na API v21.

Solução: As visualizações de 3 segundos ficaram dentro do array `actions` com o tipo `video_view`:

```
// ✗ Não funciona mais
fields: 'video_3_sec_watched_actions,...'

// ✓ Correto
const v3s = getAction(a.actions, 'video_view');
```

✕ Erro 2 — Token expirado no Vercel

```
Error validating access token: Session has expired on Sunday, 01-Mar-26 13:00:00 PST
```

Causa: O token Meta curto-prazo expirou (dura ~1–2h sem extensão, até 60d com extensão).

Solução: Gerar novo token em Graph API Explorer → estender para 60 dias → atualizar env var no Vercel → Redeploy.

Prevenção: Usar System User Token (não expira) ou criar lembrete de calendário a cada 55 dias.

✕ Erro 3 — Git push sem permissão

```
fatal: could not read Username for 'https://github.com': terminal prompts disabled
```

Causa: Git não estava configurado com credenciais no terminal.

Solução: Usar o **GitHub Desktop** para fazer commits e push sem precisar configurar credenciais no terminal.

✕ Erro 4 — Push para repositório público com token exposto

Causa: O projeto original (`mathiaschu/meta-ads-analyzer`) era público com 180 stars — enviar código com token seria expô-lo para o mundo.

Solução: Fazer fork do repo para conta própria (`grsgustao-hash/meta-ads-analyzer`) → tornar privado → só então fazer o push.

❌ Erro 5 — System User Token: "Nenhuma permissão disponível"

Causa: O System User recém-criado não estava adicionado como role em nenhum app Meta. Sem um app associado, não é possível gerar o token.

Solução: Ir em `developers.facebook.com` → app desejado → Funções do app → adicionar o System User como Admin. Depois, voltar ao Business Manager para gerar o token.

❌ Erro 6 — Nome inválido para System User

```
Você escolheu um nome de usuário do sistema inválido. Escolha outro.
```

Causa: "Dashboard API" contém espaço, o que não é permitido no nome de System Users Meta.

Solução: Usar nomes sem espaços: `dashboard-api` , `DashboardAPI` , `MetaDashboard` .

✅ Acerto — Proxy serverless para proteger o token

Em vez de expor o token no HTML (acessível por qualquer pessoa via DevTools), toda chamada passa por `/api/meta` no Vercel, que injeta o token server-side. O browser nunca vê o token.

✅ Acerto — Paginação automática com fetchPages()

A função `fetchPages(url, max)` percorre automaticamente as páginas da Meta API via `paging.next` , garantindo que todos os dados sejam carregados sem perder registros.

✅ Acerto — toProxy() para paginação segura

URLs de paginação da Meta contêm o token embutido (`?access_token=...`). A função `toProxy()` remove o token e redireciona a requisição pelo proxy, mantendo a segurança mesmo na paginação.

5 Métricas Implementadas

Todas as métricas disponíveis no dashboard e como são calculadas.

MÉTRICA	CÁLCULO	CAMPO API	COR (BOM/MÉDIO/RUIM)
Spend	Direto da API	`spend`	—
Impressions	Direto da API	`impressions`	—
CPM	spend / impressions × 1000	`cpm`	—
CTR (all)	clicks / impressions × 100	`ctr`	—
CPC Link	spend / inline_link_clicks	`inline_link_clicks`	—
Hook Rate	views_3s / impressions × 100	`actions[video_view]`	🟢 ≥30% 🟡 ≥20% 🔴 <20%
Hold Rate	views_75% / impressions × 100	`video_p75_watched_actions`	🟢 ≥20% 🟡 ≥12% 🔴 <12%
Purchases	actions[purchase]	`actions`	—
CPA	spend / purchases	`cost_per_action_type`	—
ROAS	revenue / spend	`purchase_roas`	🟢 ≥2x 🟡 ≥1x 🔴 <1x

📝 **NOTA SOBRE HOOK RATE E HOLD RATE**

Hook Rate mede o quão atraente é o início do vídeo (quantas pessoas assistiram pelo menos 3 segundos). **Hold Rate** mede o quão engajante é o conteúdo (quantas chegaram a 75% do vídeo). Ambos só fazem sentido para anúncios de vídeo — para imagens, o valor aparece como "—".

Campos solicitados à API por nível

```
// Nível campanha
'campaign_name,campaign_id,impressions,reach,frequency,
 spend,cpm,ctr,clicks,actions,action_values,
 purchase_roas,cost_per_action_type'

// Nível anúncio
```

```
'ad_name,ad_id,adset_name,adset_id,campaign_name,
 campaign_id,impressions,spend,cpm,ctr,clicks,
 inline_link_clicks,actions,action_values,purchase_roas,
 cost_per_action_type,video_p75_watched_actions'

// Nível dia (time_increment: 1)
'date_start,spend,impressions,cpm,ctr,clicks,
 actions,purchase_roas'
```

6 Como Economizar Tokens do Claude

Estratégias práticas para reduzir o consumo de contexto em projetos como este.

📋 **Compartilhe o código completo uma vez, não várias vezes**
Em vez de descrever o problema em texto, cole o bloco de código exato com erro. O Claude entende contexto técnico muito melhor com código real do que com descrições em português.

🎯 **Uma tarefa por vez, com objetivo claro**
"Adicione CPC Link, Hook Rate e Hold Rate na tabela de anúncios" é muito mais eficiente do que várias mensagens curtas explorando o que fazer. Quanto mais específico o pedido, menos idas e vindas.

🖼️ **Use prints/screenshots para mostrar erros de UI**
Um print do erro economiza muito mais tokens do que descrever o que aparece na tela. O Claude entende imagens diretamente — use isso a seu favor.

🔗 **Referencie decisões já tomadas em vez de re-explicar**
Diga "usando o padrão do proxy que já criamos" em vez de re-explicar a arquitetura toda vez. Mantenha contexto curto nas mensagens de follow-up.

📋 **Agrupe múltiplas mudanças em um único pedido**
Em vez de pedir uma métrica por mensagem ("adicione CPC Link", depois "agora Hook Rate", depois "agora Hold Rate"), agrupe tudo: "Adicione essas 3 métricas: CPC Link (spend/inline_link_clicks), Hook Rate (3s/impressions×100, verde ≥30%), Hold Rate (75%/impressions×100, verde ≥20%)".

⚡ **Dê contexto técnico direto, sem rodeios**
"O campo video_3_sec_watched_actions retornou erro 100, o que uso no lugar?" é mais eficiente do que "tô tentando fazer uma coisa aqui mas deu problema com o vídeo...". Seja direto e técnico.

🔄 **Continue conversas existentes ao invés de começar novas**
Cada nova conversa perde todo o contexto do projeto. Sempre que possível, continue a mesma sessão. Quando o contexto acabar, o Claude gera um resumo que pode ser colado no início de uma nova sessão.

📝 **Documente decisões técnicas (como este guia!)**
Com documentação, você pode briefar o Claude muito mais rápido em sessões futuras: "Leia este guia e ajude a adicionar X" — sem precisar reexplicar toda a arquitetura do zero.

📊 ESTIMATIVA DE TOKENS POR AÇÃO

AÇÃO	TOKENS APROX.	EFICIÊNCIA
Descrever problema em texto longo	~800–2000	BAIXA
Colar erro + código relevante	~300–600	ALTA
Print do erro (imagem)	~200–400	ALTA
Pedido agrupado (várias métricas)	~500–800	ALTA
Pedidos separados (uma por vez)	~1500–3000	BAIXA

7 Próximos Passos e Melhorias

Funcionalidades que podem ser adicionadas ao projeto no futuro.

Curto prazo (MVP+)

- **System User Token:** Criar token não-expirante no Meta Business Manager para nunca precisar renovar manualmente.
- **Exportar para CSV:** Botão para baixar a tabela de anúncios/campanhas como planilha.
- **Filtros na tabela:** Filtrar por campanha específica, por status, ou por range de ROAS.
- Seletor de contas já implementado nesta versão — ✅

Médio prazo

- **Comparação de períodos:** Mostrar variação % de cada métrica vs. período anterior.
- **Nível de conjunto (adset):** Adicionar aba de Conjuntos de Anúncios entre Campanhas e Anúncios.
- **Alertas automáticos:** Notificar por email/Slack quando ROAS cair abaixo de threshold.
- **Múltiplas contas em resumo:** Dashboard consolidado com todas as contas lado a lado.

Longo prazo

- **Backend com banco de dados:** Salvar histórico de dados para análises além dos 90 dias da API.
- **Auth com login:** Adicionar autenticação para que cada usuário veja só as contas dele.
- **Integração com Google Sheets:** Sincronizar dados automaticamente para análises em planilhas.

8 Checklist de Deploy

Use esta lista sempre que for fazer um novo deploy ou renovar o token.

Novo projeto do zero

- ✓ Criar pasta do projeto localmente
- ✓ Criar `api/meta.js` (proxy serverless)
- ✓ Criar `vercel.json` com rota raiz
- ✓ Criar `dashboard.html` com `apiUrl()` e `toProxy()`
- ✓ Garantir que o token NÃO está no HTML
- ✓ Criar repositório privado no GitHub
- ✓ Fazer commit e push via GitHub Desktop
- ✓ Conectar repo ao Vercel
- ✓ Adicionar `META_ACCESS_TOKEN` nas env vars do Vercel
- ✓ Fazer Redeploy após adicionar a env var
- ✓ Testar o dashboard na URL do Vercel

Renovação de token (a cada ~60 dias)

- ✓ Acessar `developers.facebook.com/tools/explorer`
- ✓ Selecionar app correto no dropdown
- ✓ Gerar token com `ads_read` + `ads_management`
- ✓ Clicar no "i" → Open in Access Token Tool → Extend (60 dias)
- ✓ Copiar o token long-lived
- ✓ No Vercel: Settings → Environment Variables → editar `META_ACCESS_TOKEN`
- ✓ Salvar nova env var
- ✓ Deployments → → Redeploy
- ⚠ Considerar System User Token para evitar essa rotina

Quando o dashboard mostrar erro

- ✓ Abrir DevTools (F12) → Console → verificar mensagem de erro
- ✓ Se "Session has expired" → renovar token (ver checklist acima)
- ✓ Se "not valid for fields param" → campo deprecated → pesquisar substituto na doc da Meta
- ✓ Se erro 500 → verificar se `META_ACCESS_TOKEN` está configurado no Vercel
- ✓ Se dados não carregam → verificar se token tem permissão `ads_read`